

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: E Batch: 3 Practical No: 6

Roll no: 63 PRN No: 12410410 Name of Student: **Pratham Shelke**

Subject Name : Deep Learning

Problem Statement

Plant diseases can significantly affect crop quality and agricultural productivity. Manual identification of diseases from leaf images is time-consuming and requires expert knowledge. This project aims to develop a **Convolutional Neural Network (CNN)** to automatically classify plant leaf images into different disease and healthy categories using the **PlantVillage dataset**.

Algorithm :-

1. **Load the PlantVillage dataset** containing 10,849 images from 38 plant disease and healthy classes.
2. **Preprocess the images** by resizing them to 128×128 pixels and normalizing pixel values to the range 0–1.
3. **Split the dataset** into training (70%), validation (15%), and testing (15%) sets.
4. **Apply data augmentation** such as rotation, zooming, shifting, and horizontal flipping to training images.
5. **Build the CNN model** using three convolutional layers followed by max-pooling layers.
6. **Add fully connected layers** with ReLU activation and dropout to reduce overfitting.
7. **Use a 38-neuron softmax output layer** to classify the images into the 38 disease/healthy categories.
8. **Compile and train the model** using the Adam optimizer and categorical cross-entropy loss for 10 epochs.
9. **Evaluate the trained model** using test accuracy, loss, precision, recall, F1-score, and a confusion matrix.
10. **Predict new leaf images** using the trained CNN and display the predicted disease class with its confidence score.

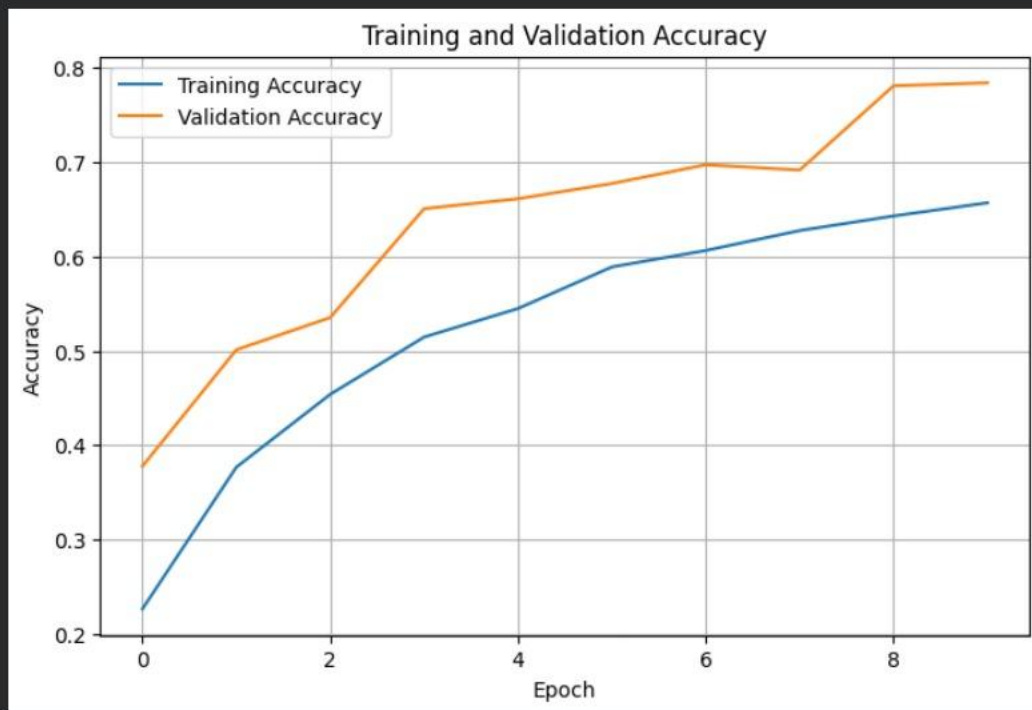
Analysis:-

1. The PlantVillage dataset contains **10,849 images belonging to 38 different classes** of healthy and diseased plants.
2. The dataset was divided into **7,577 training images, 1,612 validation images, and 1,660 testing images**.
3. Data augmentation was applied to training images to improve model generalization and reduce overfitting.
4. The CNN consists of **three convolutional layers** with max-pooling, followed by dense and dropout layers.
5. The model achieved a **validation accuracy of 78.41%** after 10 epochs.
6. On the unseen test dataset, the model achieved a **test accuracy of 77.41%** with a test loss of **0.7227**.
7. The similar validation and test accuracies indicate that the model generalizes reasonably well to unseen images.
8. Some misclassification is expected because certain plant diseases have visually similar symptoms and the dataset has **class imbalance**.
9. The model can be further improved using **transfer learning, class weighting, batch normalization, and additional training**.
10. Overall, the results demonstrate that CNNs can effectively classify plant diseases from leaf images and can serve as a foundation for an automated plant disease detection system.

OUTPUT:-

```
plt.grid(True)
```

```
plt.show()
```





[+ Code](#)[+ Text](#)

```
test_loss, test_accuracy = model.evaluate(test_generator)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

52/52 ————— **3s** 50ms/step - accuracy: 0.7741 - loss: 0.7227
Test Loss: 0.7227166295051575
Test Accuracy: 0.7740963697433472

[]



```
print(predicted_class, , predicted_class,  
print("Confidence: {:.2f}%".format(confidence))
```



Input Image



1/1 ————— 1s 881ms/step
Predicted Class: Tomato__Septoria_leaf_spot
Confidence: 38.16%

```
!pip install tensorflow pandas matplotlib seaborn scikit-learn pillow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.20.0)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: seaborn in /usr/local/lib/python3.12/dist-packages (0.13.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: pillow in /usr/local/lib/python3.12/dist-packages (11.3.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow)
Requirement already satisfied: google_pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.2)
Requirement already satisfied: protobuf>=5.28.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (5.29.6)
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.16.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.2.2)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.82.1)
Requirement already satisfied: tensorboard~=2.20.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.20.0)
Requirement already satisfied: keras>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.13.2)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.16.0)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2026.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorflow)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.10.0->tensorflow) (0.19.1)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->te
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow) (3
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow)
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorboar
Requirement already satisfied: werkzeug<1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.20.0->tensorflow)
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboard~=
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->tenso
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.10.0->ten
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3
```

```
import os

zip_path = "/val.zip"

print(os.path.exists(zip_path))
```

```
False
```

```
import zipfile
import os

zip_path = "/content/val.zip"
extract_path = "/content/plantvillage"

with zipfile.ZipFile(zip_path, "r") as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully!")
```

```
Dataset extracted successfully!
```

```
import os

print(os.listdir("/content/plantvillage"))
```

```
['val']
```

```
dataset_path = "/content/plantvillage/val"
```

```
import os

classes = sorted(os.listdir(dataset_path))

print("Number of classes:", len(classes))

for i, class_name in enumerate(classes):
    print(i, class_name)
```

```
Number of classes: 38
0 Apple__Apple_scab
1 Apple__Black_rot
2 Apple__Cedar_apple_rust
3 Apple__healthy
4 Blueberry__healthy
5 Cherry_(including_sour)__Powdery_mildew
6 Cherry_(including_sour)__healthy
7 Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot
8 Corn_(maize)__Common_rust_
9 Corn_(maize)__Northern_Leaf_Blight
10 Corn_(maize)__healthy
11 Grape__Black_rot
12 Grape__Esca_(Black_Measles)
13 Grape__Leaf_blight_(Isariopsis_Leaf_Spot)
14 Grape__healthy
15 Orange__Haunglongbing_(Citrus_greening)
16 Peach__Bacterial_spot
17 Peach__healthy
18 Pepper,_bell__Bacterial_spot
19 Pepper,_bell__healthy
20 Potato__Early_blight
21 Potato__Late_blight
22 Potato__healthy
23 Raspberry__healthy
24 Soybean__healthy
25 Squash__Powdery_mildew
26 Strawberry__Leaf_scorch
27 Strawberry__healthy
28 Tomato__Bacterial_spot
29 Tomato__Early_blight
30 Tomato__Late_blight
31 Tomato__Leaf_Mold
32 Tomato__Septoria_leaf_spot
33 Tomato__Spider_mites Two-spotted_spider_mite
34 Tomato__Target_Spot
35 Tomato__Tomato_Yellow_Leaf_Curl_Virus
36 Tomato__Tomato_mosaic_virus
37 Tomato__healthy
```

```
total_images = 0

for class_name in classes:
    class_path = os.path.join(dataset_path, class_name)

    if os.path.isdir(class_path):
        count = len([
            f for f in os.listdir(class_path)
            if f.lower().endswith((".jpg", ".jpeg", ".png"))
        ])

        print(f"{class_name}: {count}")
        total_images += count

print("\nTotal images:", total_images)
```

```
Apple__Apple_scab: 126
Apple__Black_rot: 124
Apple__Cedar_apple_rust: 55
Apple__healthy: 329
Blueberry__healthy: 300
Cherry_(including_sour)__Powdery_mildew: 210
```

```

Cherry_(including_sour)___healthy: 170
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot: 102
Corn_(maize)___Common_rust_: 238
Corn_(maize)___Northern_Leaf_Blight: 197
Corn_(maize)___healthy: 232
Grape___Black_rot: 236
Grape___Esca_(Black_Measles): 276
Grape___Leaf_blight_(Isariopsis_Leaf_Spot): 215
Grape___healthy: 84
Orange___Haunglongbing_(Citrus_greening): 1101
Peach___Bacterial_spot: 459
Peach___healthy: 72
Pepper,_bell___Bacterial_spot: 199
Pepper,_bell___healthy: 295
Potato___Early_blight: 200
Potato___Late_blight: 200
Potato___healthy: 30
Raspberry___healthy: 74
Soybean___healthy: 1018
Squash___Powdery_mildew: 367
Strawberry___Leaf_scorch: 221
Strawberry___healthy: 91
Tomato___Bacterial_spot: 425
Tomato___Early_blight: 200
Tomato___Late_blight: 381
Tomato___Leaf_Mold: 190
Tomato___Septoria_leaf_spot: 354
Tomato___Spider_mites Two-spotted_spider_mite: 335
Tomato___Target_Spot: 280
Tomato___Tomato_Yellow_Leaf_Curl_Virus: 1071
Tomato___Tomato_mosaic_virus: 74
Tomato___healthy: 318

```

Total images: 10849

```

import os
import shutil
import random

# Original dataset
source_dir = "/content/plantvillage/val"

# New dataset location
output_dir = "/content/plant_dataset"

# Split ratios
train_ratio = 0.70
validation_ratio = 0.15
test_ratio = 0.15

# Reproducibility
random.seed(42)

# Create train/validation/test folders
for split in ["train", "validation", "test"]:
    os.makedirs(os.path.join(output_dir, split), exist_ok=True)

# Process each class
for class_name in os.listdir(source_dir):

    class_source = os.path.join(source_dir, class_name)

    # Ignore files; only process directories
    if not os.path.isdir(class_source):
        continue

    # Get image files
    images = [
        f for f in os.listdir(class_source)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

    # Shuffle images
    random.shuffle(images)

    # Calculate split points
    total = len(images)

    train_end = int(total * train_ratio)
    validation_end = train_end + int(total * validation_ratio)

```



```

train_images = images[:train_end]
validation_images = images[train_end:validation_end]
test_images = images[validation_end:]

# Create class folders
for split in ["train", "validation", "test"]:
    os.makedirs(
        os.path.join(output_dir, split, class_name),
        exist_ok=True
    )

# Copy images
for image in train_images:
    shutil.copy2(
        os.path.join(class_source, image),
        os.path.join(output_dir, "train", class_name, image)
    )

for image in validation_images:
    shutil.copy2(
        os.path.join(class_source, image),
        os.path.join(output_dir, "validation", class_name, image)
    )

for image in test_images:
    shutil.copy2(
        os.path.join(class_source, image),
        os.path.join(output_dir, "test", class_name, image)
    )

print("Dataset splitting completed successfully!")

```

Dataset splitting completed successfully!

```

import os

for split in ["train", "validation", "test"]:

    split_path = os.path.join(
        "/content/plant_dataset",
        split
    )

    total = 0

    for class_name in os.listdir(split_path):

        class_path = os.path.join(
            split_path,
            class_name
        )

        if os.path.isdir(class_path):
            count = len([
                f for f in os.listdir(class_path)
                if f.lower().endswith((".jpg", ".jpeg", ".png"))
            ])

            total += count

    print(split, "images:", total)

```

```

train images: 7577
validation images: 1612
test images: 1660

```

```

train_classes = os.listdir("/content/plant_dataset/train")

print("Number of classes:", len(train_classes))

for class_name in sorted(train_classes):
    print(class_name)

```

```

Number of classes: 38
Apple__Apple_scab

```

```

Apple___Black_rot
Apple___Cedar_apple_rust
Apple___healthy
Blueberry___healthy
Cherry_(including_sour)___Powdery_mildew
Cherry_(including_sour)___healthy
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot
Corn_(maize)___Common_rust_
Corn_(maize)___Northern_Leaf_Blight
Corn_(maize)___healthy
Grape___Black_rot
Grape___Esca_(Black_Measles)
Grape___Leaf_blight_(Isariopsis_Leaf_Spot)
Grape___healthy
Orange___Haunglongbing_(Citrus_greening)
Peach___Bacterial_spot
Peach___healthy
Pepper,_bell___Bacterial_spot
Pepper,_bell___healthy
Potato___Early_blight
Potato___Late_blight
Potato___healthy
Raspberry___healthy
Soybean___healthy
Squash___Powdery_mildew
Strawberry___Leaf_scorch
Strawberry___healthy
Tomato___Bacterial_spot
Tomato___Early_blight
Tomato___Late_blight
Tomato___Leaf_Mold
Tomato___Septoria_leaf_spot
Tomato___Spider_mites Two-spotted_spider_mite
Tomato___Target_Spot
Tomato___Tomato_Yellow_Leaf_Curl_Virus
Tomato___Tomato_mosaic_virus
Tomato___healthy

```

```

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Image dimensions
IMG_SIZE = (128, 128)

# Number of images processed at once
BATCH_SIZE = 32

# Dataset paths
train_path = "/content/plant_dataset/train"
validation_path = "/content/plant_dataset/validation"
test_path = "/content/plant_dataset/test"

# Training data augmentation
train_datagen = ImageDataGenerator(
    rescale=1./255,
    rotation_range=20,
    width_shift_range=0.1,
    height_shift_range=0.1,
    zoom_range=0.2,
    horizontal_flip=True
)

# Validation and test data: only normalization
validation_datagen = ImageDataGenerator(
    rescale=1./255
)

test_datagen = ImageDataGenerator(
    rescale=1./255
)

# Load training images
train_generator = train_datagen.flow_from_directory(
    train_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical"
)

# Load validation images

```

```
validation_generator = validation_datagen.flow_from_directory(
    validation_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical"
)

# Load test images
test_generator = test_datagen.flow_from_directory(
    test_path,
    target_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    class_mode="categorical",
    shuffle=False
)

print("\nNumber of classes:", train_generator.num_classes)
```

Found 7577 images belonging to 38 classes.
 Found 1612 images belonging to 38 classes.
 Found 1660 images belonging to 38 classes.

Number of classes: 38

```
print("Training images:", train_generator.samples)
print("Validation images:", validation_generator.samples)
print("Test images:", test_generator.samples)
print("Number of classes:", train_generator.num_classes)
```

Training images: 7577
 Validation images: 1612
 Test images: 1660
 Number of classes: 38

```
class_indices = train_generator.class_indices

for class_name, index in class_indices.items():
    print(index, "->", class_name)
```

```
0 -> Apple___Apple_scab
1 -> Apple___Black_rot
2 -> Apple___Cedar_apple_rust
3 -> Apple___healthy
4 -> Blueberry___healthy
5 -> Cherry_(including_sour)___Powdery_mildew
6 -> Cherry_(including_sour)___healthy
7 -> Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot
8 -> Corn_(maize)___Common_rust_
9 -> Corn_(maize)___Northern_Leaf_Blight
10 -> Corn_(maize)___healthy
11 -> Grape___Black_rot
12 -> Grape___Esca_(Black_Measles)
13 -> Grape___Leaf_blight_(Isariopsis_Leaf_Spot)
14 -> Grape___healthy
15 -> Orange___Haunglongbing_(Citrus_greening)
16 -> Peach___Bacterial_spot
17 -> Peach___healthy
18 -> Pepper,_bell___Bacterial_spot
19 -> Pepper,_bell___healthy
20 -> Potato___Early_blight
21 -> Potato___Late_blight
22 -> Potato___healthy
23 -> Raspberry___healthy
24 -> Soybean___healthy
25 -> Squash___Powdery_mildew
26 -> Strawberry___Leaf_scorch
27 -> Strawberry___healthy
28 -> Tomato___Bacterial_spot
29 -> Tomato___Early_blight
30 -> Tomato___Late_blight
31 -> Tomato___Leaf_Mold
32 -> Tomato___Septoria_leaf_spot
33 -> Tomato___Spider_mites Two-spotted_spider_mite
34 -> Tomato___Target_Spot
35 -> Tomato___Tomato_Yellow_Leaf_Curl_Virus
36 -> Tomato___Tomato_mosaic_virus
37 -> Tomato___healthy
```

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

```
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    Flatten,
    Dense,
    Dropout
)

# Create CNN model
model = Sequential([

    # Convolutional Layer 1
    Conv2D(
        32,
        (3, 3),
        activation="relu",
        input_shape=(128, 128, 3)
    ),

    # Pooling Layer 1
    MaxPooling2D(pool_size=(2, 2)),

    # Convolutional Layer 2
    Conv2D(
        64,
        (3, 3),
        activation="relu"
    ),

    # Pooling Layer 2
    MaxPooling2D(pool_size=(2, 2)),

    # Convolutional Layer 3
    Conv2D(
        128,
        (3, 3),
        activation="relu"
    ),

    # Pooling Layer 3
    MaxPooling2D(pool_size=(2, 2)),

    # Convert feature maps to 1D
    Flatten(),

    # Fully connected layer
    Dense(
        128,
        activation="relu"
    ),

    # Dropout to reduce overfitting
    Dropout(0.5),

    # Output layer
    Dense(
        38,
        activation="softmax"
    )
])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 126, 126, 32)	896
max_pooling2d (MaxPooling2D)	(None, 63, 63, 32)	0
conv2d_1 (Conv2D)	(None, 61, 61, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 30, 30, 64)	0
conv2d_2 (Conv2D)	(None, 28, 28, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 128)	0
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 128)	3,211,392
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 38)	4,902

Total params: 3,309,542 (12.62 MB)

Trainable params: 3,309,542 (12.62 MB)

Non-trainable params: 0 (0.00 B)

```
model.compile(
    optimizer="adam",
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

print("Model compiled successfully!")
```

Model compiled successfully!

```
history = model.fit(
    train_generator,
    epochs=10,
    validation_data=validation_generator
)
```

```
Epoch 1/10
237/237 ————— 55s 207ms/step - accuracy: 0.2266 - loss: 2.9936 - val_accuracy: 0.3778 - val_loss: 2.3353
Epoch 2/10
237/237 ————— 43s 179ms/step - accuracy: 0.3768 - loss: 2.3026 - val_accuracy: 0.5012 - val_loss: 1.8153
Epoch 3/10
237/237 ————— 43s 180ms/step - accuracy: 0.4540 - loss: 1.9644 - val_accuracy: 0.5354 - val_loss: 1.6402
Epoch 4/10
237/237 ————— 42s 175ms/step - accuracy: 0.5147 - loss: 1.7125 - val_accuracy: 0.6507 - val_loss: 1.1577
Epoch 5/10
237/237 ————— 40s 167ms/step - accuracy: 0.5449 - loss: 1.5649 - val_accuracy: 0.6613 - val_loss: 1.1064
Epoch 6/10
237/237 ————— 40s 168ms/step - accuracy: 0.5890 - loss: 1.4033 - val_accuracy: 0.6774 - val_loss: 1.0661
Epoch 7/10
237/237 ————— 39s 164ms/step - accuracy: 0.6064 - loss: 1.3427 - val_accuracy: 0.6973 - val_loss: 0.9848
Epoch 8/10
237/237 ————— 39s 166ms/step - accuracy: 0.6276 - loss: 1.2479 - val_accuracy: 0.6917 - val_loss: 0.9648
Epoch 9/10
237/237 ————— 39s 164ms/step - accuracy: 0.6430 - loss: 1.1712 - val_accuracy: 0.7810 - val_loss: 0.7204
Epoch 10/10
237/237 ————— 41s 172ms/step - accuracy: 0.6570 - loss: 1.1329 - val_accuracy: 0.7841 - val_loss: 0.7043
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

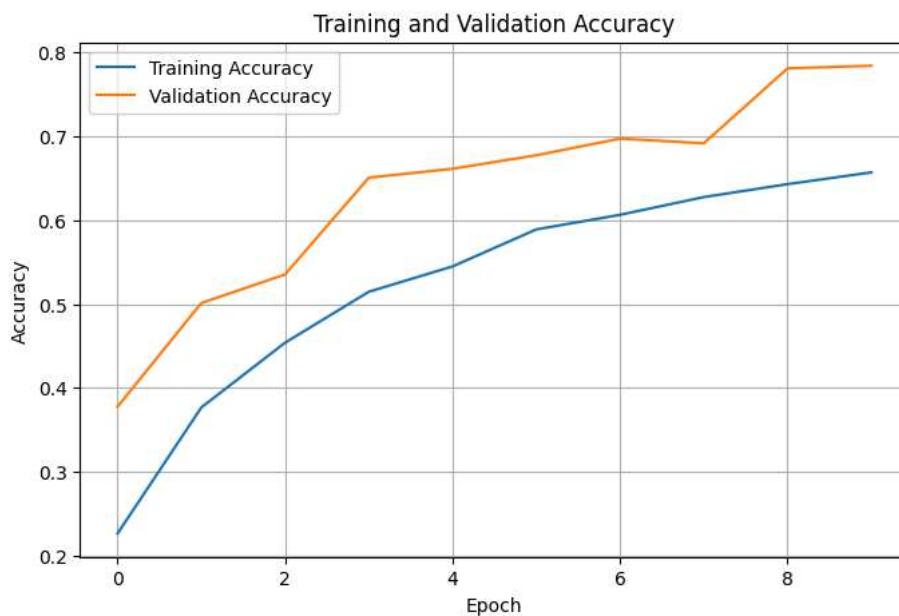
plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)
```

```
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Training and Validation Accuracy")

plt.legend()
plt.grid(True)

plt.show()
```



```
plt.figure(figsize=(8, 5))

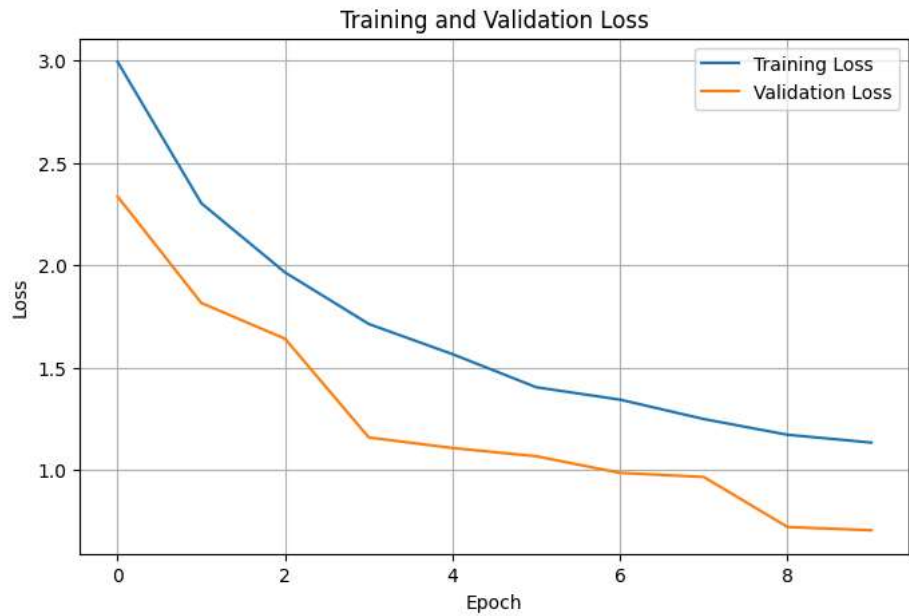
plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Training and Validation Loss")

plt.legend()
plt.grid(True)

plt.show()
```



```
test_loss, test_accuracy = model.evaluate(test_generator)

print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
```

52/52 ————— 3s 50ms/step - accuracy: 0.7741 - loss: 0.7227
Test Loss: 0.7227166295051575
Test Accuracy: 0.7740963697433472

```
import numpy as np

predictions = model.predict(test_generator)

predicted_classes = np.argmax(
    predictions,
    axis=1
)

true_classes = test_generator.classes

class_names = list(
    test_generator.class_indices.keys()
)

print("Predictions completed!")
print("Number of predictions:", len(predicted_classes))
```

52/52 ————— 3s 41ms/step

Predictions completed!
Number of predictions: 1660

```
from sklearn.metrics import classification_report

report = classification_report(
    true_classes,
    predicted_classes,
    target_names=class_names
)

print(report)
```

	precision	recall	f1-score	support
Apple___Apple_scab	1.00	0.20	0.33	20
Apple___Black_rot	0.58	0.55	0.56	20
Apple___Cedar_apple_rust	1.00	0.11	0.20	9
Apple___healthy	0.72	0.66	0.69	50
Blueberry___healthy	0.62	1.00	0.76	45
Cherry_(including_sour)___Powdery_mildew	0.75	0.66	0.70	32
Cherry_(including_sour)___healthy	0.75	0.56	0.64	27
Corn_(maize)___Cercospora_leaf_spot Gray_leaf_spot	0.62	0.50	0.55	16

Corn_(maize)___Common_rust_	0.92	0.97	0.95	37
Corn_(maize)___Northern_Leaf_Blight	0.73	0.71	0.72	31
Corn_(maize)___healthy	1.00	1.00	1.00	36
Grape___Black_rot	0.90	0.50	0.64	36
Grape___Esca_(Black_Measles)	0.71	0.95	0.82	42
Grape___Leaf_blight_(Isariopsis_Leaf_Spot)	0.96	0.76	0.85	33
Grape___healthy	0.79	0.79	0.79	14
Orange___Haunglongbing_(Citrus_greening)	0.96	0.97	0.97	166
Peach___Bacterial_spot	0.83	0.70	0.76	70
Peach___healthy	0.91	0.83	0.87	12
Pepper,_bell___Bacterial_spot	0.86	0.61	0.72	31
Pepper,_bell___healthy	0.65	0.71	0.68	45
Potato___Early_blight	0.62	0.97	0.75	30
Potato___Late_blight	0.75	0.20	0.32	30
Potato___healthy	0.00	0.00	0.00	5
Raspberry___healthy	0.67	0.50	0.57	12
Soybean___healthy	0.82	0.94	0.87	154
Squash___Powdery_mildew	0.74	0.98	0.85	56
Strawberry___Leaf_scorch	0.77	0.88	0.82	34
Strawberry___healthy	0.92	0.80	0.86	15
Tomato___Bacterial_spot	0.75	0.69	0.72	65
Tomato___Early_blight	0.67	0.20	0.31	30
Tomato___Late_blight	0.60	0.62	0.61	58
Tomato___Leaf_Mold	0.88	0.24	0.38	29
Tomato___Septoria_leaf_spot	0.48	0.74	0.58	54
Tomato___Spider_mites_Two-spotted_spider_mite	0.59	0.78	0.67	51
Tomato___Target_Spot	0.58	0.69	0.63	42
Tomato___Tomato_Yellow_Leaf_Curl_Virus	0.94	0.91	0.93	162
Tomato___Tomato_mosaic_virus	1.00	0.58	0.74	12
Tomato___healthy	0.81	0.98	0.89	49
accuracy			0.77	1660
macro avg	0.76	0.67	0.68	1660
weighted avg	0.79	0.77	0.76	1660

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels: {'True Class: 100'}
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels: {'True Class: 100'}
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is ill-defined for classes in true labels: {'True Class: 100'}
_warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))

```

```

from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(
    true_classes,
    predicted_classes
)

plt.figure(figsize=(18, 15))

sns.heatmap(
    cm,
    cmap="Blues",
    xticklabels=class_names,
    yticklabels=class_names
)

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix - Plant Disease CNN")

plt.xticks(rotation=90)
plt.yticks(rotation=0)

plt.tight_layout()
plt.show()

```



```
import os

os.makedirs("/content/models", exist_ok=True)

model.save("/content/models/plant_disease_cnn.keras")

print("Model saved successfully!")
```

Model saved successfully!
Cherry (including sour) Powdery mildew -

```
from tensorflow.keras.preprocessing import image
import numpy as np
import matplotlib.pyplot as plt

# Path to your new image
img_path = "/tomato_leaf.webp"

# Load and resize image
img = image.load_img(
    img_path,
    target_size=(128, 128)
)

# Display image
plt.figure(figsize=(5, 5))
plt.imshow(img)
plt.axis("off")
plt.title("Input Image")
plt.show()

# Convert image to array
img_array = image.img_to_array(img)

# Normalize pixel values
img_array = img_array / 255.0

# Add batch dimension
img_array = np.expand_dims(img_array, axis=0)

# Make prediction
prediction = model.predict(img_array)

# Get predicted class
predicted_index = np.argmax(prediction[0])

predicted_class = class_names[predicted_index]

# Get confidence
confidence = prediction[0][predicted_index] * 100

print("Predicted Class:", predicted_class)
print("Confidence: {:.2f}%".format(confidence))
```

tomato_tomato_mosaic_virus
Input Image